

Sapphire Local Runtime Benchmark

A reproducible local measurement, not a cross-language, GPU, or cluster benchmark.

This manual describes the current local implementation, tested behavior, and honest boundaries of Sapphire. It is intentionally detailed so the document remains readable and useful beyond a short summary.

Benchmark definition

The included benchmark focuses on a simple repeated parse-and-interpret workload instead of claiming supercomputer-scale performance. The idea is to provide a reproducible local measurement that can be run repeatedly on a given machine with a known Python version and environment. This keeps the benchmark useful for regression checking and local comparison without overstating global performance.

The reference workload is a 100-item loop with arithmetic accumulation. The measured result is deterministic for a given test machine, but the speed numbers are not meant to be interpreted as general language rankings or proof of a production-grade runtime. They are evidence about the local implementation under one environment.

Reference run

On a reference machine running Python 3.12.10, the 100-item loop benchmark produced 4950 as the result, with a median of 0.21095 ms, a minimum of 0.2023 ms, and a maximum of 0.7817 ms. These values are useful as a data point, but they should be interpreted as local timing numbers tied to a specific hardware and software environment.

The benchmark intentionally avoids claiming a wider scaling story. It does not claim a token-per-second metric, a GPU throughput number, or a cluster efficiency score, because those workloads are not part of the benchmark itself. Those claims would be misleading without the underlying infrastructure, hardware, and validation.

How to reproduce

The project includes the benchmark script under `benchmarks/benchmark_runtime.py`, and the easiest way to reproduce it is to run that script from the repository root. The measurement is intentionally simple, and repeated runs can be used to compare performance before and after code changes. Machine load, CPU frequency scaling, Python version, and operating system differences all affect the results.

When interpreting results, it is important to remember that parsing, evaluation, and scheduling overhead are all included in the visible numbers. A benchmark like this is a useful local signal, but it is not a substitute for workload-specific testing in production, GPU-rich environments, or distributed clusters.

Benchmark ethics and scope

The project's benchmark philosophy is conservative. It tries to provide evidence that the language can execute a specific local workload reliably, but it avoids unsupported claims such as cross-language rankings or large-scale throughput. This is a necessary safeguard: users can compare behavior under a controlled local environment without mistaking a simple benchmark for a broad systems claim.

This keeps the benchmark honest and useful. It gives developers a practical measurement they can run, verify, and compare, while still telling the truth about what it does not cover.

Version 1.0.7. This document is intentionally longer than a brief summary to provide context, operational detail, and practical caveats for local use. Values and capabilities depend on the machine, software environment, and installed dependencies.